LAPORAN PRAKTIKUM
ANALISIS KOMPLEKSITAS
DAN ALGORITMA

Disusun Oleh:

Ratisya Dwi Putri

NIM	**: 2025573010105**
KELAS	**: TI.1D**
PDODI	**: TEKNIK INFORMATIKA**
DOSEN PENGAJAR	**: M. REZA ZULFAN, SST., M.Sc**

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan/praktikum ini dengan baik. Laporan ini disusun sebagai bagian dari pembelajaran mengenai analisis efisiensi algoritma dalam pemrograman.

Dalam dunia pemrograman, kemampuan untuk memahami dan menganalisis efisiensi algoritma sangatlah penting. Tidak hanya sekadar membuat program yang berjalan, tetapi juga memastikan bahwa program tersebut optimal dalam penggunaan waktu dan memori. Oleh karena itu, pada materi ini dibahas konsep dasar seperti *Time Complexity*, *Space Complexity*, serta penggunaan notasi Big O sebagai alat ukur performa algoritma.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan dalam bidang pemrograman.

Buketrata,….2026

Ratisya Dwi Putri

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan Penulisan 1
BAB II 2
LANDASAN TEORI 2
2.1 Pengertian Efisiensi Algoritma 2
2.2 Time Complexity dan Space Complexity 2
2.3 Penggunaan Notasi Big O 2
2.4 Menentukan Kompleksitas Loop 2
2.5 Membandingkan Performa Algoritma 3
2.6 Mengidentifikasi Algoritma yang Tidak Efisien 3
BAB III 4
LANGKAH KERJA 4
3.1 Alat dan Bahan 4
3.2 Langkah Kerja 4
3.3 Hasil Praktikum 7
2.4Submit ke GitHub 13
BAB IV 14
PENUTUP 14
4.1 Kesimpulan 14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang pesat menyebabkan jumlah data yang diolah semakin besar dan kompleks. Dalam kondisi tersebut, pemilihan algoritma yang tepat menjadi sangat penting agar program dapat berjalan secara efisien dan tidak membebani sistem.

Efisiensi algoritma tidak hanya dilihat dari hasil akhir, tetapi juga dari seberapa cepat algoritma tersebut menyelesaikan proses dan seberapa besar memori yang digunakan. Tanpa analisis yang baik, sebuah program berpotensi mengalami penurunan kinerja ketika jumlah data meningkat.

Oleh karena itu, diperlukan pemahaman mengenai analisis efisiensi algoritma, termasuk konsep Time Complexity, Space Complexity, serta notasi Big O. Dengan memahami hal tersebut, pengembang dapat menghindari penggunaan algoritma yang kurang efisien dan mampu meningkatkan performa program secara keseluruhan.

1.2 Rumusan Masalah

1. Apa pengertian efisiensi algoritma?
2. Apa yang dimaksud dengan Time Complexity dan Space Complexity?
3. Bagaimana cara menggunakan notasi Big O dalam analisis algoritma?
4. Bagaimana menentukan kompleksitas dari suatu loop?
5. Bagaimana cara membandingkan performa algoritma?
6. Bagaimana mengidentifikasi algoritma yang tidak efisien?

1.3 Tujuan Penulisan

1. Menjelaskan konsep efisiensi algoritma.
2. Memahami Time Complexity dan Space Complexity.
3. Menguasai penggunaan notasi Big O.
4. Menganalisis kompleksitas kode program.
5. Membandingkan kinerja algoritma secara praktis.
6. Meningkatkan kemampuan optimasi algoritma.

BAB II

LANDASAN TEORI

2.1 Pengertian Efisiensi Algoritma

Efisiensi algoritma adalah ukuran kinerja suatu algoritma dalam menyelesaikan masalah dengan mempertimbangkan waktu eksekusi dan penggunaan memori. Algoritma yang efisien mampu menghasilkan output yang benar dengan waktu yang lebih cepat dan penggunaan sumber daya yang lebih sedikit. Hal ini sangat penting terutama ketika menangani data dalam jumlah besar.

2.2 Time Complexity dan Space Complexity

- **Time Complexity** adalah ukuran yang menunjukkan seberapa lama waktu yang dibutuhkan algoritma untuk berjalan berdasarkan ukuran input (n).
- **Space Complexity** adalah ukuran yang menunjukkan seberapa besar memori yang digunakan oleh algoritma selama proses eksekusi.

Kedua konsep ini digunakan untuk mengevaluasi performa algoritma sehingga dapat dipilih solusi yang paling optimal.

2.3 Penggunaan Notasi Big O

Notasi Big O digunakan untuk menggambarkan tingkat pertumbuhan waktu atau ruang yang dibutuhkan oleh algoritma terhadap ukuran input. Notasi ini membantu dalam membandingkan efisiensi berbagai algoritma.

Beberapa contoh notasi Big O:

- **O(1)** → waktu konstan
- **O(log n)** → logaritmik
- **O(n)** → linear
- **O(n log n)** → lebih efisien dibanding kuadratik
- **O(n^2)** → kuadratik (kurang efisien untuk data besar)

2.4 Menentukan Kompleksitas Loop

Kompleksitas suatu loop dapat ditentukan dari jumlah iterasi yang dilakukan:

- **Loop tunggal** yang berjalan sebanyak n kali memiliki kompleksitas **O(n)**
- **Nested loop (loop bersarang)** yang masing-masing berjalan n kali memiliki kompleksitas **O(n^2)**

- Loop yang mengurangi input secara bertahap (misalnya dibagi dua setiap iterasi) memiliki kompleksitas **O(log n)**

Analisis ini membantu dalam memahami bagian mana dari program yang paling mempengaruhi kinerja.

2.5 Membandingkan Performa Algoritma

Performa algoritma dapat dibandingkan secara empiris dengan mengukur waktu eksekusi. Dalam JavaScript, hal ini dapat dilakukan menggunakan fungsi Date.now().

Contoh sederhana:

- Jalankan dua algoritma berbeda
- Catat waktu mulai dan selesai
- Bandingkan hasil waktu eksekusi

Dengan cara ini, dapat diketahui algoritma mana yang lebih cepat dalam praktik.

2.6 Mengidentifikasi Algoritma yang Tidak Efisien

Algoritma yang tidak efisien biasanya memiliki ciri-ciri sebagai berikut:

- Menggunakan terlalu banyak nested loop
- Melakukan perhitungan berulang yang tidak diperlukan
- Tidak memanfaatkan struktur data yang tepat

Untuk memperbaikinya dapat dilakukan dengan:

- Mengurangi jumlah perulangan
- Mengoptimalkan logika program
- Menggunakan algoritma yang lebih efisien seperti pencarian atau pengurutan yang lebih baik

BAB III

LANGKAH KERJA

3.1 Alat dan Bahan

Alat

1. Komputer atau Laptop
2. Akses Internet

Bahan

1. GitBash
2. Visual Studio Code

3.2 Langkah Kerja

1. Ketik cd/c
2. Masuk ke folder: cd nama folder
3. Masuk ke Repository: cd 2025573010105
4. Buat folder praktikum-5, mkdir praktikum-5
5. cd praktikum-5
6. npm init -y

```
LENOVO@LAPTOP-Q1HJ91QN MINGW64 ~
$ cd /c

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c
$ cd "/c/Praktikum Struktur Data dan Algoritma"

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma
$ cd 2025573010105_strukturdataalgoritma

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma (main)
$ mkdir praktikum-5

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma (main)
$ cd praktikum-5

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ npm init -y
Wrote to C:\Praktikum Struktur Data dan Algoritma\2025573010105_strukturdataalgoritm
a\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

7. Buat file: touch 01-intro-kompleksitas.js di GitBash
8. Lalu buka file tersebut dengan VS Code
9. Lalu ketik node 01-intro-kompleksitas.js, di GitBash
10. Hasilnya:

```
LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1)   jumlahkanRumus  : 0 ms
O(n)   jumlahkanLinear : 1 ms
O(n^2) cariPasangan    : 15 ms

Hasil sama? true
```

11. Buat file: touch 02-destructuring-spread.js di GitBash
12. Lalu buka file tersebut dengan VS Code
13. Lalu ketik node 02-destructuring-spread.js, di GitBash
14. Hasilnya:

```
LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ touch 02-destructuring-spread.js

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ node 02-destructuring-spread.js
=== O(1) - selalu cepat ===
10
true

=== O(log n) - Binary Search ===
Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) - Linear Search ===
Max dari 1000 elemen: 999.9799584709712

=== O(n^2) - Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2^n) - Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2^n):
84 ms

Waktu fib(35) O(n) memoization:
0 ms
```

15. Buat file: touch 03-space-complexity.js di GitBash
16. Lalu buka file tersebut dengan VS Code, kerjakan latihan
17. Lalu ketik node 03-space-complexity.js, di GitBash
18. Hasilnya:

```
LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ touch 03-space-complexity.js

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : undefined
Elemen unik: 7
```

19. Buat file: touch 04-refactor.js di GitBash
20. Lalu buka file tersebut dengan VS Code
21. Lalu ketik node 04-refactor.js, di GitBash
22. Hasilnya:

```
LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n^2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 2, b: 2, c: 1, d: 1 }
```

23.

```
LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ touch latihan-1.js
```

3.3 Hasil Praktikum

Latihan 1

1. Buat file baru latihan-1.js

   latihan-1.js

2. Kerjakan Latihan pada VS Code

```javascript
function fungsiA(n){
    return n * 2;
} //Big 0: 0(1) - Hanya satu operasi aritmatika

function fungsiB(n){
    for (let i=0; i<n; i++){
        for (let j=0; j<n; j++){
            console.log(i,j);
        }
    }
} //Big 0: 0(n^2) - perulangan bersarang n * n

function fungsiC(n){
    for (let i=1; i<n; i*=2){
        console.log(i);
    }
} //Big 0: 0(log n) - Iterasi meningkat secara eksponensial (i*2)

function fungsiD(n){
    const arr = Array.from({length: n}, (_, i) => i);
    arr.forEach(x => {
        arr.forEach(y => {
            arr.forEach(z => {
                console.log(x,y,z);
            });
        });
    });
} // Big O: O(n^3) - Tiga lapis perulangan pada array berukuran n
```

```javascript
} // Big O: O(n^3) - Tiga lapis perulangan pada array berukuran n

function hitungKompleksitas(n, fn){
    const startTime = Date.now();
    fn(n);
    const endTime = Date.now();
    const duration = endTime - startTime;

    console.log(`Fungsi: ${fn.name} | n: ${n} | Waktu: ${duration}ms`);
}

const n = 1000;

console.log("Hasil pengukuran waktu");
hitungKompleksitas(n, fungsiA);
hitungKompleksitas(n, fungsiB);
hitungKompleksitas(n, fungsiC);
hitungKompleksitas(n, fungsiD);
```

Latihan 3

1. Buka File 03-space-complexity.js, kerjakan Latihan di VS Code

Program dan Output tugas:

2. Masuk Kembali ke Gitbash, ketik cd/c
3. Masukkan folder cd"/c/Praktikum Struktur Data dan Algoritma"
4. Lalu ke repository cd 2025573010105_strukturdataalgoritma
5. Lalu masuk ke folder praktikum-5, cd praktikum-5
6. Lalu ketik mkdir tugas
7. Touch tugas/tugas-01.js tugas/tugas-2.js
8. Hasilnya:

```
LENOVO@LAPTOP-Q1HJ91QN MINGW64 ~
$ cd /c

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c
$ cd "/c/Praktikum Struktur Data dan Algoritma"

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma
$ cd 2025573010105_strukturdataalgoritma

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma (main)
$ cd praktikum-5

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ mkdir tugas

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2
```

9. Lalu buka file tugas-1.js di VS Code kerjakan tugas:
   - Tugas 1

```javascript
function intersectionLambat(arr1, arr2) {
    const result = [];
    for (let i = 0; i < arr1.length; i++) {
        for (let j = 0; j < arr2.length; j++) {
            if (arr1[i] === arr2[j]) {
                result.push(arr1[i]);
                break;
            }
        }
    }
    return result;
}

function intersectionCepat(arr1, arr2) {
    const set = new Set(arr2);
    return arr1.filter(x => set.has(x));
}

function kelompokAnagram(arr) {
    const din = {};
    for (let kata of arr) {
        const key = kata.split('').sort().join('');
        if (!din[key]) {
            din[key] = [];
        }
        din[key].push(kata);
    }
    return Object.values(din);
}

function tripletLambat(arr) {
    const n = arr.length;
```

```
function tripletLambat(arr) {
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            for (let k = 0; k < n; k++) {
                if (arr[i] ** 2 + arr[j] ** 2 === arr[k] ** 2) {
                    return true;
                }
            }
        }
    }
    return false;
}

function tripletCepat(arr) {
    arr = arr.map(x => x * x).sort((a, b) => a - b);

    for (let c = arr.length - 1; c >= 2; c--) {
        let left = 0;
        let right = c - 1;

        while (left < right) {
            if (arr[left] + arr[right] === arr[c]) {
                return true;
            } else if (arr[left] + arr[right] < arr[c]) {
                left++;
            } else {
                right--;
            }
        }
    }
    return false;
}
```

```
> (2) [2, 3]
> (2) [2, 3]
> (3) [Array(3), Array(2), Array(1)]
true
true
intersectionLambat: 7 ms
intersectionCepat: 1 ms
tripletLambat: 8 ms
tripletCepat: 2 ms
```

10. Kemudian buka file Tugas-2.js di VS Code, Kerjakan

- Tugas 2

```
//konstan
function fn_O1(n){
    return n + 1;
}
//linear
function fn_On(n){
    let sum = 0;
    for(let i = 0; i <= n; i++){
        sum++;
    }
    return sum;
}
//linear mitm 0 n Log n
function fn_OnLogn(n){
    let count =0;
    for(let i = 0; i <=n; i++){
        for(let j = 1; j <=n; j++);
    }
}
//kuadratik
function fn_On2(n){
    let count = 0;
    for(let i = 0; i <= n; i++){
        for(let j = 1; j <=n; j++){
            count++;
        }
    }
    return count;
}
function bechmarkSemua(ukuranData){
    ukuranData.forEach(n => {
        console.log(`\nUkuran Data: ${n}`);
```

```javascript
function bechmarkSemua(ukuranData){
    ukuranData.forEach(n => {
        console.log(`\nUkuran Data: ${n}`);
        console.log('\n===konstan===');
        console.time('O(1)');
        fn_O1(n);
        console.timeEnd('O(1)');

        console.log('\n===Linear===');
        console.time('O(n)');
        fn_On(n);
        console.timeEnd('O(n)');

        console.log('\n===Linear mithm===');
        console.time('O(n log n)');
        fn_OnLogn(n);
        console.timeEnd('O(n log n)');

        console.log('\n===kuadratik===');
        console.time('O(n2)');
        fn_On2(n);
        console.timeEnd('O(n2)');

    });

}
bechmarkSemua([100, 500, 1000, 5000, 10000]);
```

```
Ukuran Data: 100

===konstan===
O(1): 0.066162109375 ms
O(1): 0.153ms

===Linear===
O(n): 0.067138671875 ms
O(n): 0.099ms

===Linear mithm===
O(n log n): 0.345947265625 ms
O(n log n): 0.426ms

===kuadratik===
O(n2): 0.35302734375 ms
O(n2): 0.453ms

Ukuran Data: 500

===konstan===
O(1): 0.057861328125 ms
O(1): 0.104ms

===Linear===
O(n): 0.02880859375 ms
O(n): 0.061ms
```

```
===Linear mithm===
O(n log n): 2.39404296875 ms
O(n log n): 3.264ms

===kuadratik===
O(n2): 0.7509765625 ms
O(n2): 0.927ms

Ukuran Data: 1000

===konstan===
O(1): 0.0400390625 ms
O(1): 0.163ms

===Linear===
O(n): 0.1259765625 ms
O(n): 0.293ms

===Linear mithm===
O(n log n): 0.699951171875 ms
O(n log n): 0.951ms

===kuadratik===
O(n2): 2.05908203125 ms
O(n2): 3.103ms

Ukuran Data: 5000

===konstan===
O(1): 0.017822265625 ms
```

```
O(1): 0.233ms

===Linear===
O(n): 0.1728515625 ms
O(n): 0.542ms

===Linear mithm===
O(n log n): 63.81982421875 ms
O(n log n): 64.406ms

===kuadratik===
O(n2): 74.02001953125 ms
O(n2): 74.113ms

Ukuran Data: 10000

===konstan===
O(1): 0.012939453125 ms
O(1): 0.05ms

===Linear===
O(n): 0.028076171875 ms
O(n): 0.055ms

===Linear mithm===
O(n log n): 191.333984375 ms
O(n log n): 191.474ms

===kuadratik===
O(n2): 179.01708984375 ms
O(n2): 179.115ms
```

2.4 Submit ke GitHub

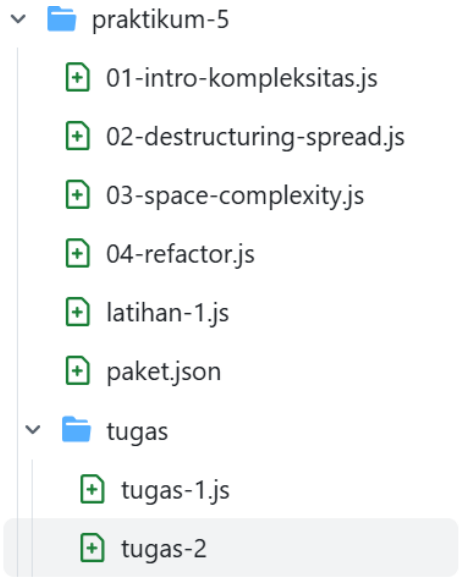
1. Kembali ke room folder-Repository dan ketik cd..
2. Ketik git add praktikum-5/
3. Lalu ketik git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
4. Hasilnya:

```
LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma/praktikum-5 (main)
$ cd ..

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010_strukturdataalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        praktikum-5/

nothing added to commit but untracked files present (use "git add" to track)

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by C
RLF the next time Git touches it

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma (main)
$ git commit-m "Praktikum 5: Analisis Kompleksitas Algoritma"
git: 'commit-m' is not a git command. See 'git --help'.

The most similar command is
        commit-tree

LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main 0a48f1f] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 439 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-destructuring-spread.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2
```

5. git push origin main
6. hasilnya:

```
LENOVO@LAPTOP-Q1HJ91QN MINGW64 /c/Praktikum Struktur Data dan Algoritma/202557301010
5_strukturdataalgoritma (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 4 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 5.13 KiB | 1.28 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ratisyaratisya4/202557301010_strukturdataalgoritma.git
   bff68a7..0a48f1f  main -> main
```

7. Stelah di submit ke GitHub:
8. Hasilnya:



BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan pembahasan yang telah dilakukan, dapat disimpulkan bahwa efisiensi algoritma merupakan hal yang sangat penting dalam pengembangan program, terutama ketika menangani data dalam jumlah besar. Dengan memahami konsep *Time Complexity* dan *Space Complexity*, seorang programmer dapat mengevaluasi kinerja algoritma secara lebih terukur.

Penggunaan notasi Big O membantu dalam menggambarkan tingkat efisiensi algoritma serta memudahkan perbandingan antar algoritma. Selain itu, analisis terhadap struktur seperti loop dan nested loop sangat berpengaruh dalam menentukan kompleksitas suatu program.

Pengujian secara empiris juga menunjukkan bahwa algoritma dengan kompleksitas lebih rendah memiliki performa yang lebih baik dibandingkan algoritma dengan kompleksitas tinggi. Oleh karena itu, kemampuan dalam mengidentifikasi dan mengoptimasi algoritma menjadi sangat penting untuk menghasilkan program yang efisien.